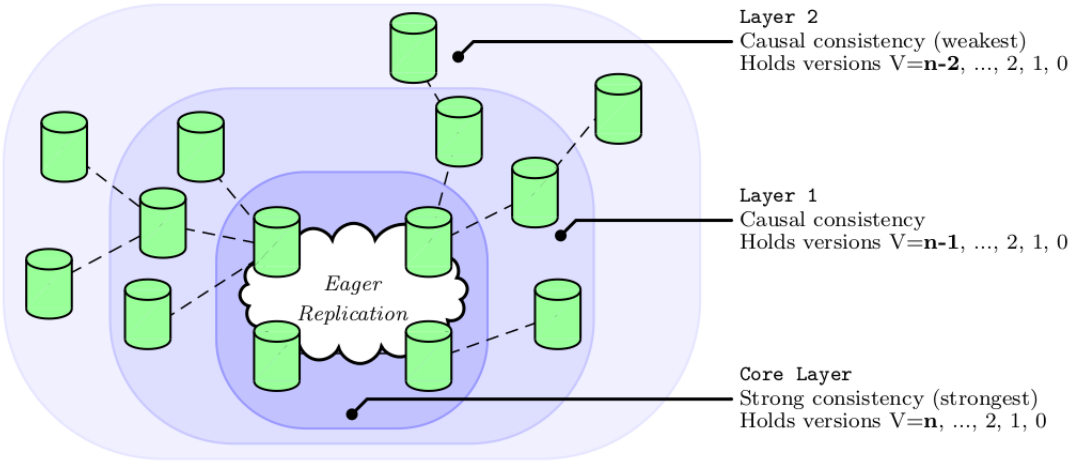
EXERCISE 3: EPIDEMIC DATA REPLICATION

Learning Objectives
By the end of this exercise, you should be able to:

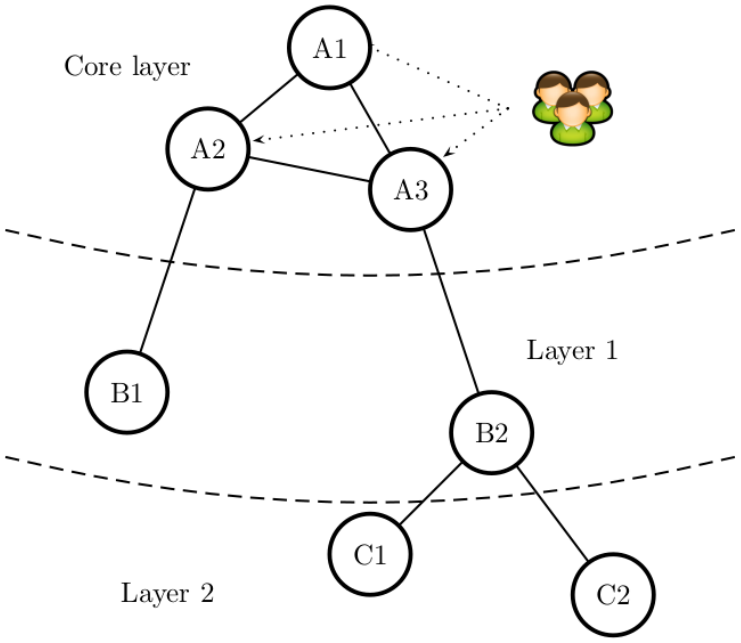
- Understand the importance of data replication and its strategies.
- Implement a multi-layered distributed system.

Exercise statement
Design and implement a distributed application in charge of replicating the data in an epidemic manner to achieve a multi-versioned data architecture similar to the following:



In this system, the nodes that belong to the *core layer* have the most modern versions of the object. However, nodes belonging to the second layer have the oldest versions. Note that (1) the different nodes of a same layer will always have the same version (maybe old, but the same!), (2) by time = ∞ all nodes will converge towards the same version if the *core layer* does not propagate more updates (aka *eventual consistency*), (3) the clients issue update transactions to the *core layer*, and (4) read-only transactions can performed on any layer.
Specifically, we want to build a system like that of the figure *below* in which there are 7 nodes and 3 layers. the characteristics of the system are the following:

1. Communication between nodes must be via sockets or gRPC.
2. The *core layer* uses *update everywhere*, *active*, and *eager replication*.
3. The nodes B1 and B2 from the second layer, receive the data every 10 updates (*lazy*) through passive replication, *primary backup*.
4. Nodes C1 and C2 receive the data every 10 seconds (lazy) through passive replication, *primary backup*.
5. Each node must maintain a local text file the log of all the versions it keeps.
6. Clients launch transactions specified in a local file. Transactions can be of two types:
   - ***Read-only***: `b<f>, r(30), r(49), r(69), c`. Where <f> is an integer that is 0, 1 or 2 and indicates the layer on which the transaction must be executed.
   - ***Not read-only***: `b, r(12), w(49,53), r(69), c`. These transactions will always be executed in one of the nodes of the *core layer*.
7. The data of the replicas must be monitored in real time through *web sockets*. Create a web application for this purpose.



Exercise guidelines

- The source code and the report of the exercise must be submitted separately (two files) in eStudy delivery task.
- The exercise can be developed in any programming language of your choice, as long as it follows the general guidelines.
- Communication can be developed through sockets or gRPC protocol.
- The web client must at least offer system monitoring functionalities. Extending the functionalities is always welcomed.

Collaboration policy

- Everyone can work solo or in a team of two people for this assignment.
- Groups are strongly encouraged to discuss the details about the project with their partners.

> All of the source code of the project must be your own. You may not copy code from other sources from peers or on the web. Plagiarism will not be tolerated. See La Salle URL's Plagiarism Regulation for additional information.